

Final Report

<Full title of project>

Karl Samuel Kassis

Submitted in accordance with the requirements for the degree of
<Name of degree programme>

<Session/Academic year>

<Module code and name>

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (DD/MM/YY)
<Example> Scanned participant consent forms	PDF file / file archive	Uploaded to Minerva (DD/MM/YY)
<Example> Link to online code repository	URL	Sent to supervisor and assessor (DD/MM/YY)
<Example> User manuals	PDF file	Sent to client and supervisor (DD/MM/YY)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) _____

Summary

The primary aim of this project was to design, implement, and optimize a high-performance computational engine and Artificial Intelligence (AI) agent for the abstract board game Nonaga. To achieve an advanced level of play and facilitate rapid algorithmic experimentation, the project was structured around three core technical objectives:

1. **High-Performance Engine Development:** To engineer a highly optimized rule engine and state generator to maximize state evaluations per second while avoiding the performance bottlenecks of object-oriented representations.
2. **Adversarial AI Implementation:** To develop a robust AI opponent utilizing Minimax.
3. **Automated Parameter Optimization:** To construct a Genetic Algorithm framework that utilizes self-play tournaments to autonomously tune the AI overcoming the limitations of manual heuristic configuration.

The resulting system successfully integrates these components with an interactive PyGame graphical interface, providing both a seamless user experience for human-vs-AI gameplay and a powerful backend for continued heuristic optimization and profiling.

Acknowledgements

<The page should contain any acknowledgements to those who have assisted with your work. Where you have worked as part of a team, you should, where appropriate, reference to any contribution made by other to the project.>

Note that it is not acceptable to solicit assistance on ‘proof reading’ which is defined as the “the systematic checking and identification of errors in spelling, punctuation, grammar and sentence construction, formatting and layout in the text”; see

https://www.leeds.ac.uk/secretariat/documents/proof_reading_policy.pdf

Contents

1	Introduction and Background Research	1
1.1	Project Aim and Context	1
1.2	Introduction to Nonaga and the Computational Problem	1
1.2.1	What is Nonaga?	1
1.3	Board State Representation	2
1.3.1	Object-Oriented Representation	2
1.3.2	Bitboard Representation	2
1.4	Adversarial Search	3
1.4.1	Monte Carlo Tree Search	3
1.4.2	Minimax	4
1.5	Heuristic Evaluation and Parameter Optimization	5
1.5.1	Automated Parameter Tuning	6
2	Methodology	8
2.1	Core Engine Design: Representation and Computation	8
2.1.1	Board State Representation	8
2.2	Artificial Intelligence and Search Strategy	8
2.2.1	Adversarial Search Algorithm	8
2.2.2	Evaluation Function	9
2.3	Evaluation Function Optimization	9
2.3.1	Optimization of the Evaluation Function	9
2.4	System Architecture and Technology Stack	9
2.4.1	Language Selection and Hybrid Integration	9
2.5	Project Management	10
2.5.1	Iterative Development Phases	10
2.5.2	Scope Management	11
3	Implementation and Validation	12
3.1	The High-Performance Game Engine (C/Cython)	12
3.1.1	Bitboard Representation: Memory and Architecture	12
3.1.2	Optimized Move Generation Algorithms	14
3.2	AI and Search Implementation	15
3.2.1	Zobrist Hashing for Transposition Table	15
3.2.2	Transposition Table Integration and Best Move Ordering	15
3.2.3	Parameterized Evaluation Function	16
3.3	The Genetic Algorithm Framework	17
3.3.1	GA Architecture and Encoding	17
3.3.2	Fitness Evaluation via Tournaments	17
3.3.3	Selection, Crossover, and Mutation Operators	17

3.4	Testing Approaches	18
3.4.1	Manual State Verification via Custom UI	18
3.4.2	Execution Time Profiling Methodology	18
3.4.3	Empirical Validation Of The Evolutionary Heuristic Optimization	19
4	Results, Evaluation and Discussion	21
4.1	Engine Performance Evaluation	21
4.1.1	Legacy Python to C/Cython Migration	21
4.1.2	Legacy Engine Scaling Failure at Depth 3	22
4.1.3	Discussion	22
4.2	Final AI Search Performance	22
4.2.1	Minimax Enhancements Effectiveness	22
4.2.2	Transposition Table Hit-Rate Analysis	24
4.2.3	Combined Optimisation: Final Engine Throughput	25
4.2.4	Discussion	26
4.3	Genetic Algorithm Optimisation Results	26
4.4	Evaluation Summary	26
	Appendices	31
A	Self-appraisal	31
A.1	Critical self-evaluation	31
A.1.1	Project Scope and Time Management	31
A.2	Personal reflection and lessons learned	31
A.2.1	Lessons Learned in Cross-Language Integration	31
A.3	Legal, social, ethical and professional issues	31
A.3.1	Legal issues	31
A.3.2	Social issues	31
A.3.3	Ethical issues	31
A.3.4	Digital Game Addiction and Gambling Considerations	31
A.3.5	Professional issues	31
B	External Material	32
C	Nonaga's Branching Factor Calculation	33
D	Brute-Force Feasibility Analysis	35
E	Benchmark Implementation	36
E.1	Benchmarking Harnesses Variations	36

Chapter 1

Introduction and Background Research

1.1 Project Aim and Context

The primary aim of this project is to architect and develop a high-performance computer program capable of playing the board game Nonaga at a highly competitive level. While playing a board game is intuitively simple for humans, teaching a machine to do so requires solving complex computational challenges. Therefore, this project serves as both an interactive application and an experimental platform for artificial intelligence.

To achieve this overarching aim, the project focuses on several core objectives, designed to tackle the problem from both a user-experience and a high-level computational perspective:

- **Engine Optimization (The Board Representation):** Before an AI can play, it must understand the board state efficiently. A major objective is to design a highly optimized internal representation of the game. This foundational layer must process rules and generate possible moves almost instantaneously, ensuring the AI can operate at high speeds without performance bottlenecks.
- **Adversarial Search (Looking Ahead):** The AI needs the ability to predict future outcomes. This objective involves implementing a strategic decision-making algorithm that allows the system to simulate thousands of potential future board states. By evaluating these outcomes, the system can quickly discard mathematically poor moves and select the most advantageous ones within a reasonable timeframe.
- **Evaluation Optimization (Developing Intuition):** The AI must be able to judge whether a specific, mid-game board position is advantageous. To enhance this "intuition," the project implements an automated learning framework. By having the AI play thousands of simulated matches, the system systematically evolves and improves its own evaluation parameters without relying on human intervention.
- **Interactive Client Development (User Interface):** To make the engine accessible and testable, the final objective is building a responsive Graphical User Interface (GUI). This interface not only allows human users to play against the AI, but also serves as a debugging tool to analyzing the AI's decision-making process.

Furthermore, to the best of our knowledge, there is currently no published academic literature addressing Nonaga.

1.2 Introduction to Nonaga and the Computational Problem

1.2.1 What is Nonaga?

Nonaga is an abstract strategy board game created by Viktor Bautista i Roca that opposes two players [2]. The primary objective is straightforward: a player wins by maneuvering their three

individual pieces so that they connect and form a single continuous group.

The game is played on a dynamically shifting board composed of 19 independent hexagonal tiles. The structure of a turn consists of two distinct mandatory phases. First, a player must slide one of their pieces in a straight line; the piece cannot stop voluntarily and must continue moving until it collides with another piece or reaches the edge of the board. Second, the player must alter the playing area itself by removing an unoccupied tile from the outer edge of the board and reattaching it to a different valid location on the perimeter.

Although these mechanics are intuitive for a human player, this two-phase turn structure introduces severe computational complexity. Because a single turn requires both a piece movement and a tile movement which alters the structure of the board, the number of possible legal moves available at any given moment, is exceptionally high (estimated at 440 in Appendix C). Moreover, the state of the board requires continuous recomputation; between each turn, the legality of moving any specific tile must be dynamically evaluated based on the connectivity of its immediate neighbors. For applications that require rapidly simulating thousands of games, such as training artificial opponents to play the game and automatically evolving their strategies over time, the continuous recomputation needed can become an important overhead.

1.3 Board State Representation

1.3.1 Object-Oriented Representation

The most straightforward approach for board state representation is Object-Oriented Design (OOD), which instantiates discrete game entities as classes. While this method accelerates initial development and aligns with human conceptual models of a board game, OOD struggles to efficiently utilize computing resources in performance-critical applications [30]. Because objects are dynamically allocated, they become scattered across system memory. During the millions of rapid state evaluations required by an Artificial Intelligence (AI) search algorithm, this lack of spatial locality leads to severe CPU cache misses and memory latency, severely bottlenecking engine performance [1].

1.3.2 Bitboard Representation

On the opposite, modern high-performance engines rely on Data-Oriented Design (DOD), which prioritizes memory contiguity and hardware-level efficiency. By representing the game state as a series of 64-bit integers, known as "bitboards", the engine can map boolean states (e.g., piece presence, color, or valid tiles) directly to individual bits. Studies have shown that contiguous, bit-level representations can achieve speedups of over 13x compared to equivalent OOP architectures by maximizing cache efficiency and allowing parallel evaluations via bitwise logical operations (AND, OR, XOR) [30].

While bitboards are the de facto standard in computer chess, they present unique implementation paradigms that must be carefully evaluated. As Segundo et al. (2006) note, while bitboard state transitions can theoretically operate 64 times faster than array encodings, poorly optimized bit-manipulation can introduce significant overhead, making the underlying

mathematical model critical [24]. Furthermore, generalizing bitboards to non-standard geometries requires careful mathematical mapping, as outlined by Browne (2014) [8].

Evaluating Bitboard Architectures for Sliding Pieces

A fundamental challenge in bitboard design is move generation, which is broadly categorized into "leaping" and "sliding" mechanics. Leaping pieces (such as a Knight in chess) have static attack patterns relative to their coordinate, requiring a simple array lookup:

`Attacks[coordinate]`.

Conversely, sliding pieces, which move continuously along an axis until obstructed by the board edge or another piece, require dynamic calculation based on current board occupancy.

Historically, computing these sliding rays using step-by-step bitwise shifts was computationally expensive. To mitigate this, two primary architectures used for chess have dominated engine design:

- **Rotated Bitboards:** An older, slower technique that maintains multiple redundant bitboards rotated along different axes (e.g., 90° , 45° , 315°). This transforms diagonal and vertical rays into contiguous horizontal memory segments, allowing fast array lookups at the cost of maintaining multiple synchronized board states.
- **Direct Lookup via Perfect Hashing (Magic Bitboards):** The modern, memory intensive, state-of-the-art methodology formalized by Tannous (2007) [28]. This technique isolates the pieces along a slider piece's ray (blocking their path) and multiplies this occupancy bitboard by a pre-calculated 64-bit integer (the "Magic Number"). Following a bit-shift, this operation yields a dense, perfect hash index used to fetch the bitboard corresponding to the coordinates visible to the sliding piece. This reduces complex ray-casting and collision detection to a handful of CPU cycles: `Mask` $\rightarrow$ `Multiply` $\rightarrow$ `Shift` $\rightarrow$ `Array Lookup`.

1.4 Adversarial Search

Nonaga is formalized as a two-player, deterministic, zero-sum game of perfect information. Identifying optimal strategies within such environments traditionally necessitates traversing the state-space via adversarial search algorithms. Currently, the two preeminent paradigms for state-space evaluation in game artificial intelligence (AI) are Monte Carlo Tree Search (MCTS) and the Minimax algorithm [9].

1.4.1 Monte Carlo Tree Search

MCTS has demonstrated profound empirical success in modern game AI, particularly within domains characterized by massive branching factors, such as Go [9]. The algorithm evaluates game states by incrementally constructing an asymmetric search tree and simulating thousands of randomized playouts (rollouts) to terminal states. A distinct advantage of MCTS is its ability to estimate state values without relying on complex, domain-specific heuristic evaluation functions. Conversely, MCTS imposes substantial memory overhead to maintain the search

tree, and the stochastic nature of its rollouts necessitates considerable computational time to converge on statistically robust estimations [19].

1.4.2 Minimax

Minimax is a foundational game-theoretic algorithm that performs a recursive, depth-first exploration of the game tree. In its theoretical form, searching exhaustively to terminal nodes, it guarantees the computation of a deterministic Nash equilibrium [19]. However, naive Minimax rapidly becomes computationally intractable for games with substantial branching factors (b) and depths (d), exhibiting an exponential time complexity of $O(b^d)$.

Consequently, practical implementations are restricted to a constrained depth limit, substituting terminal game states with a static, expert-crafted heuristic evaluation function. It is critical to note that under these constraints, the algorithm no longer guarantees a true Nash equilibrium; rather, it computes an approximated optimal strategy. The reliability of this approximation is strictly bounded by the fidelity of the heuristic function, making the engine susceptible to the horizon effect if the heuristic incorrectly evaluates the true strength of a leaf node. To mitigate the exponential explosion of the search tree and allow for deeper, more accurate heuristic evaluations, Minimax is typically augmented with algorithmic enhancements.

Alpha-Beta Pruning

Alpha-Beta pruning is an optimization technique that mathematically guarantees the return of the identical optimal decision as standard Minimax, but dramatically reduces the requisite number of node evaluations. It achieves this by maintaining upper and lower bounds (alpha and beta) and discarding branches mathematically proven to yield worse outcomes than previously evaluated paths. In the best case, Alpha-Beta pruning effectively halves the exponent of the algorithmic complexity from $O(b^d)$ to $O(b^{d/2})$, yielding an effective branching factor of $\sqrt{b}$ [21]. In the worst case, it evaluates as many nodes as standard Minimax. In highly optimized implementations, advanced engines leverage this method to reduce their effective branching factor to nearly 1.5, permitting significantly deeper searches within identical time constraints [9].

Transposition Table

A Transposition Table (TT) is a dynamic hash table utilized to cache the heuristic evaluations and search depths of previously visited game states [16]. Due to the overlapping nature of game trees, identical states can often be reached via different permutations of moves (transpositions). The TT allows the algorithm to bypass redundant subtree recalculations, fundamentally enabling efficient Iterative Deepening [27]. To minimize the computational cost of state serialization and mitigate hash collisions, the TT typically employs Zobrist Hashing, generating a near-unique integer for a given board configuration via XOR operations [31]. However, the overhead associated with TT maintenance, specifically state hashing, memory access latency, and collision resolution, must be critically balanced [7]. Poorly optimized TT maintenance overhead can exhibit performance degradation that eclipses the computational benefits of pruning depending on the nature of the search space and specific implementation.

Move Ordering and Iterative Deepening

Move Ordering is a heuristic technique that dictates the sequence in which child nodes are generated and evaluated by the Alpha-Beta algorithm [21, 5]. The primary objective is to evaluate the strongest candidate moves first, establishing rigid alpha and beta bounds early in the search, thereby maximizing the frequency of subsequent node cutoffs [5]. This enhancement is highly synergistic with Iterative Deepening, which conducts sequential depth-limited searches. By caching the principal variation (the sequence of best moves) discovered at depth $i - 1$, the algorithm can prioritize exploring these nodes first at depth i [23, 6].

Additionally, the ordering of sibling nodes can be achieved through either static domain-dependent evaluations or dynamic search-based heuristics. While computationally inexpensive static evaluations can be applied to rank nodes based on game-specific features (such as capture priority), they introduce algorithmic overhead. To bypass this computational bottleneck, advanced engines heavily rely on dynamic, domain-independent techniques such as the Killer Heuristic and the History Heuristic [21]. These methods prioritize moves that have successfully provoked beta-cutoffs in parallel branches of the search tree, exploiting the localized relevance of tactical threats. The efficacy of Move Ordering is intrinsically linked to the accuracy of these combined techniques; if the prescribed order fails to reflect the true optimal sequence, the engine incurs sorting and evaluation overhead without the corresponding pruning benefits. Consequently, the computational cost of the ordering mechanism must remain strictly minimal to ensure a net-positive performance yield [5, 6].

1.5 Heuristic Evaluation and Parameter Optimization

Because exhaustive search to terminal nodes is computationally intractable for games with substantial branching factors, the Minimax algorithm must terminate at a constrained depth limit. At these leaf nodes, a static heuristic evaluation function is deployed to approximate the true game-theoretic utility of the state.

Linear Evaluation Functions

The foundational approach to static evaluation, originally formalized by Claude Shannon [25], is the linear evaluation function. This model quantifies a game state by extracting a vector of discrete, domain-specific features (e.g., piece mobility, structural connectivity, or center control) and calculating their weighted sum. Mathematically, it is expressed as $E(s) = \sum_{i=1}^n w_i f_i(s)$, where f_i represents the extracted feature and w_i is its corresponding static weight. The primary architectural advantage of a linear function is its extreme computational efficiency. Because it relies exclusively on scalar multiplication and addition, it introduces minimal overhead to the engine, maximizing the nodes-per-second (NPS) throughput. This high-speed evaluation is highly synergistic with Alpha-Beta pruning, allowing the search algorithm to reach significantly deeper horizons within identical time constraints [23, 5]. However, its fundamental limitation is its inability to natively model non-linear interactions; if the strategic value of one feature is strictly dependent on the presence of another, a purely linear combination cannot inherently capture this dependency [26, 15].

Neural Networks and Non-Linear Models

To address the limitations of linear models, modern game-playing architectures leverage Deep Neural Networks (DNNs) to evaluate board states. As demonstrated by deep reinforcement learning systems like AlphaZero [26], DNNs can successfully approximate highly complex, non-linear relationships between board features, yielding superhuman evaluative accuracy. However, this accuracy incurs a massive computational penalty. The requisite tensor operations and matrix multiplications are exceptionally slow compared to basic arithmetic, dramatically reducing the maximum depth a purely CPU-bound Minimax engine can reach. To bridge this computational divide, state-of-the-art engines frequently employ Efficiently Updatable Neural Networks (NNUE) [15]. NNUEs utilize shallow network architectures where only the specific neurons affected by a move are recalculated, offering a compromise between the rich, non-linear accuracy of a neural network and the necessary execution speed of a traditional linear heuristic.

1.5.1 Automated Parameter Tuning

While linear evaluation functions offer optimal search speeds, their efficacy is entirely predicated on the configuration of their weight vector $(w_1 \dots w_n)$ [25]. Because manual tuning relies on inherently subjective domain expertise and becomes impractical in multi-dimensional spaces, discovering the optimal weights is formalized as a continuous or discrete optimization problem. Various automated paradigms exist to navigate this search space.

Hill Climbing

Local search metaheuristics, such as Hill Climbing, iteratively perturb individual weights and adopt them if benchmarks yield higher win rates. While computationally lightweight, this method is notoriously susceptible to premature convergence, frequently trapping the optimizer in local optima within the complex fitness landscapes of game-tree heuristics [21, 5].

Simulated Annealing

Temporal Difference

Alternatively, Temporal Difference (TD) learning updates weights dynamically based on the temporal sequence of game states, using gradient descent to minimize the error between consecutive evaluations. This method, famously utilized in Arthur Samuel’s seminal checkers program [22], allows the engine to learn incrementally from continuous self-play without requiring explicit datasets. However, TD learning requires mathematically rigid formulations of the reward signal and learning rate; if misconfigured, the weight updates can become highly unstable [26].

Standard Evolutionary Algorithms

Genetic Algorithms (GAs) [11] provide a robust, population-based alternative for global optimization. Inspired by biological evolution, a standard GA maintains a diverse population of candidate weight vectors (chromosomes). Through successive generations, these vectors are evaluated via fitness functions, typically structured as round-robin or Swiss-system

tournaments, where the highest-performing configurations are selected for reproduction. By applying stochastic operators such as crossover (recombining weights from two parents) and mutation (randomly altering specific weights), GAs excel at broad exploration of the search space, mitigating the local optima traps that plague local search methods. Despite their exploratory strengths, standard GAs exhibit structural limitations when optimizing highly correlated parameters. In many board games, heuristic features are interdependent; for example, the optimal weight for piece mobility may strictly depend on the weight assigned to board centrality. This phenomenon, known as epistasis [11], can cause standard, position-independent crossover operators to continuously destruct highly effective combinations of weights (building blocks) during reproduction, severely bottlenecking the algorithm's optimization trajectory.

Linkage Learning in Genetic Algorithms

To address the destructive nature of standard crossover operations in epistatic environments [11], modern evolutionary computation introduces Linkage Learning [17]. This advanced paradigm augments the traditional GA by actively identifying dependencies and correlations between specific genes within the chromosome. Instead of relying on blind, uniform crossover, Linkage Learning algorithms often employ underlying machine learning models or statistical dependency matrices [18] to map the structure of high-performing genetic building blocks. During reproduction, these linked clusters of weights are preserved and transferred as a single unit, ensuring that synergistic parameter interactions are maintained across generations. While Linkage Learning significantly improves the quality and robustness of the generated offspring in complex search spaces, it introduces substantial algorithmic complexity and computational overhead to the generational update phase, requiring rigorous mathematical profiling to ensure viability [29].

Chapter 2

Methodology

2.1 Core Engine Design: Representation and Computation

2.1.1 Board State Representation

A fundamental design choice in the development of the game engine was the internal representation of the Nonaga board. Traditional implementations of hexagonal board games often rely on 2D arrays or object-oriented graph structures. However, these approaches incur significant overhead and CPU cache-miss penalties during the millions of node evaluations required by deep adversarial search.

To achieve maximum computational throughput, a Bitboard architecture was implemented. By mapping the 19-cell hexagonal grid of Nonaga to an array of 64-bit integers, the entire game state is compressed. Consequently, complex operations such as move generation, tile movability status, and win-condition validation are reduced to highly efficient, CPU-native bitwise operations (AND, OR, XOR) that evaluate the board state in parallel against precomputed bitmasks. While the application of bitboards is well-documented for games like Chess [28], adapting this architecture to Nonaga’s unique, dynamic topology represented a calculated engineering risk due to the lack of existing literature. However, the theoretical execution speed justified the increased developmental complexity.

2.2 Artificial Intelligence and Search Strategy

2.2.1 Adversarial Search Algorithm

While Monte Carlo Tree Search (MCTS) is a popular choice for games with high branching factors, an enhanced Minimax algorithm was selected for this project. Because Nonaga is heavily geometric, a highly accurate heuristic evaluation function could be reliably constructed. Leveraging this domain knowledge via Minimax was estimated to provide a more deterministic and robust early-game performance than the randomized rollouts of standard MCTS.

The primary decision-making agent utilizes this Minimax algorithm, significantly optimized via Alpha-Beta Pruning, Iterative Deepening, and a Transposition Table (TT) backed by Zobrist Hashing. Furthermore, the TT does more than just eliminate the redundant computation of identical board states; it serves as a highly efficient First-Move Ordering mechanism maximizing the efficiency of the Alpha-Beta pruning process. Instead of incurring the heavy computational overhead of dynamically sorting all possible legal moves at every node, the engine utilizes a Hash Move heuristic. By saving the hash of the best move encountered during the previous Iterative Deepening depth and retrieving it via the TT to evaluate first, the engine captures the vast majority of pruning benefits with a fraction of the algorithmic complexity.

2.2.2 Evaluation Function

Because terminal win-states cannot be reached at shallow depths, the Minimax algorithm relies heavily on a parameterized heuristic evaluation function. This seamlessly integrates with Nonaga highly geometric nature. Each parameter can be multiplied to a geometric feature of the board that can be efficiently derived from the bitboard. Furthermore, this approach requires very little development time while still yielding substantial results making it a better fit for this project compared to the implementation of neural networks and non linear methods.

2.3 Evaluation Function Optimization

2.3.1 Optimization of the Evaluation Function

To systematically optimize the weights of these parameters, a Genetic Algorithm (GA) framework was selected over traditional hill-climbing methodologies.

A Random K-Opponent Tournament Scheduler was engineered, forcing genomes to compete against each other in self-play to determine their fitness. A unique challenge in optimizing heuristics for adversarial games is that fitness is entirely relative; a gene's performance is strictly dependent on the quality of its opponents. To mitigate this noise, the K parameter (number of opponents) must be set as high as computationally acceptable to ensure a statistically significant fitness score.

It was recommended to benchmark the GA's convergence against an absolute "apex gene" found via brute-force search over a restricted domain (1–6 per parameter) [12]. However, calculations revealed this to be unfeasible due to the estimated execution time and heavily neighborhood-dependent nature of the fitness landscape (further detailed in Appendix D). While advanced evolutionary strategies, specifically Linkage Learning, were proposed as a potential solution to this landscape dependency [12], their implementation was ultimately deemed out of scope. The core objective of the GA was to tune the evaluation function parameters to a highly performant standard, which the implemented baseline achieved successfully (As discussed in Section 4.3). For that reason, development resources were strategically reallocated to engine polishing and system optimization rather than over-engineering the evolutionary framework.

2.4 System Architecture and Technology Stack

2.4.1 Language Selection and Hybrid Integration

The overarching architectural goal was to deliver an interactive desktop client without compromising the extreme execution speeds required by the AI engine. Pure Python, while excellent for rapid UI prototyping via the PyGame library, was deemed fundamentally unsuitable for the core algorithmic engine. The Python Global Interpreter Lock (GIL) and the inherent overhead of dynamic object handling introduce severe bottlenecks when executing computationally dense bitwise operations and recursive tree searches.

To resolve this, a hybrid technology stack was engineered. The low-level Bitboard logic and Minimax core were strictly implemented in standard C, granting precise control over memory

allocation and maximum instruction optimization. Python was retained exclusively for high-level orchestration, user interface rendering, and the GA tournament scheduling. Cython was utilized as the critical bridging layer, allowing the C-compiled backend to be exposed as native Python modules. This multi-language architecture cleanly separates the presentation layer from the algorithmic core, guaranteeing that both the interactive gameplay and the offline GA training pipelines execute against the exact same high-performance ruleset without rule drift or latency.

Python/C was selected over C++, Java, or Julia to maximize development efficiency through existing writing proficiency; alternative languages offered insufficient benefits to justify their learning curves.

2.5 Project Management

Throughout the project lifecycle, `git` was employed as the primary version control system to manage the evolving codebase, track granular changes, and maintain a secure repository. Initially, a rigid, linear project plan was formulated in the Project Outline and Specification; however, early implementation phases revealed that this structure was overly abstracted and practically restrictive. The original plan operated on the flawed assumption of strict modular independence. In reality, the high degree of uncertainty in the exact implementation methods rendered the strict Waterfall approach implied in the plan nonviable.

Consequently, the project management strategy shifted to an adaptive, feature-driven Agile methodology. Development was anchored by Test-Driven Development (TDD) principles to ensure the stability of the features implemented. The workflow followed a strict iterative loop: a target feature was engineered, rigorously verified against its functional requirements, and fully stabilized before continuous research dictated the selection and prioritization of the subsequent development target.

2.5.1 Iterative Development Phases

Because the project required continuous research and adaptation, the development lifecycle was structured around flexible objectives rather than strict time-boxed intervals. Each phase focused on delivering a functional, verifiable component of the overarching system:

- **Phase 1: Core Mechanics and Visualization:** The initial focus was establishing a ground-truth representation of the game. The core rules of Nonaga were implemented in Python, paired with an interactive PyGame graphical user interface. By the end of this phase, human players could execute a valid game from start to finish, establishing a reliable baseline for future AI integration.
- **Phase 2: Adversarial Search Integration:** With a valid state representation in place, the focus shifted to algorithmic decision-making. By the end of this phase, the Minimax algorithm, enhanced with alpha-beta pruning, was successfully implemented to evaluate game trees.
- **Phase 3: Low-Level Optimization:** The objective of this phase was to minimize execution latency, thereby incrementally increasing the permissible search depth of the AI.

By the end of this phase, the AI achieved a stable search depth of 3 within a pragmatic execution timeframe (a few seconds per move). This phase encompassed the language migration of performance-critical loops from pure Python to Cython/C, alongside the integration of bitboards, Transposition Tables, and move-ordering heuristics.

- **Phase 4: Diagnostic Tooling and Verification:** Custom diagnostic UI tools (detailed in Section 3.4.1) were developed to manually verify the functional validity of the underlying engine modifications. This phase was introduced to establish a robust debugging infrastructure, facilitating the immediate isolation of logical regressions and minimizing subsequent debugging overhead as the architecture grew in complexity.
- **Phase 5: Evolutionary Tuning and Profiling:** The final phase focused on maximizing the AI's heuristic accuracy and computational efficiency. By the end of this phase, a modular Genetic Algorithm framework was constructed to automate the tuning of evaluation weights, resulting in a highly proficient artificial opponent.

This phase-driven approach provided the necessary flexibility to adapt to emergent technical challenges. Rather than adhering strictly to predetermined milestones, development iteratively cycled back to previous phases to address critical bottlenecks such as returning to core optimization (Phase 3) when subsequent heuristic tuning (Phase 5) revealed underlying performance constraints. Ultimately, the priority was to engineer a robust, high-performing AI, justifying deviations from the initial, rigid project specification.

2.5.2 Scope Management

As advised during the interim progress review, a pragmatic approach to scope management was adopted [12]. Rather than expanding the project horizontally to encompass distributed cloud computing or widespread multi-platform deployment, development deliberately prioritized the deep optimization and polishing of the existing core architecture. This targeted focus ensured that the underlying AI engine and the local tuning frameworks achieved a rigorous standard of performance and stability without stretching the project's resources too thin.

Chapter 3

Implementation and Validation

3.1 The High-Performance Game Engine (C/Cython)

3.1.1 Bitboard Representation: Memory and Architecture

Unlike traditional board game implementations that use Object-Oriented coordinate structures, this engine relies on a "bitboard" architecture. In this paradigm, the board is not stored as a collection of objects; instead, the hexagonal play space is flattened, and every single hexagonal cell is mapped to a single bit (0 or 1) inside a standard 64-bit integer as seen in Figure 3.1a. In the integer used to store the board, each bit represents a certain coordinate. In a game like chess, where the board is static, this integer contains one bit for each square on the board. Doing the same for Nonaga, where the storing integer contains as many bits as there are tiles that form the board, would fail given the dynamic nature of the Nonaga board. To fix this issue, the integer is set to have an arbitrary number of bits such that it can express a large array of positions in which the whole Nonaga game will be played which will be referred to as the play-field. The play-field was chosen to be a hexagon of radius 10, containing 441 cells, so that it's size doesn't limit game-play (Illustrated in Figure 3.1b). Conveniently, the entire board state can be encoded using an array of seven 64-bit integers ($7 \times 64 = 448$). Since the play-field is finite it theoretically bounds the number of possible moves, in the event that the board drifts to one of its edges. In practice, this situation is very unlikely to happen through intelligent or random play and was never encountered through all the games simulated during the development of this project.

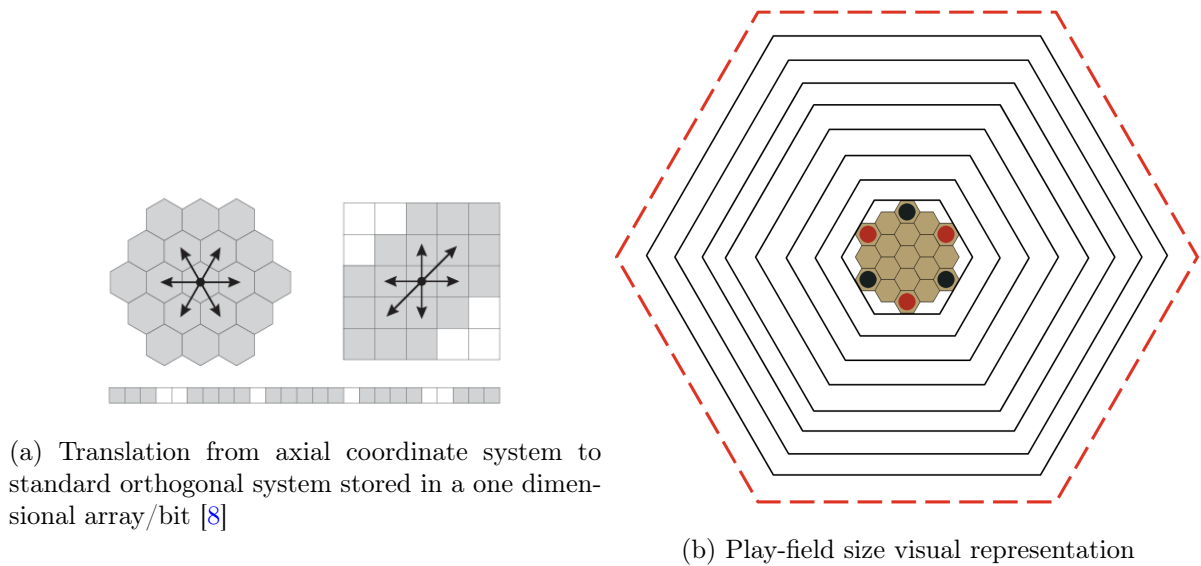


Figure 3.1

To map the hexagonal board geometry into a linear bit sequence, the engine employs a coordinate transformation from an axial hex-coordinate system (q, r) to a flattened

one-dimensional index [8]. Because axial coordinates can be negative, a constant spatial offset (`BOARD_OFFSET = 10`) is applied to shift the origin, ensuring all values fall within a non-negative range. The hexagonal space is subsequently embedded into a 2D square grid defined by `BOARD_WIDTH = 10 + 1 + 10 = 21`. The linear index is then computed using row-major flattening:

$$\text{index} = (q + 10) + (r + 10) \times 21$$

This index directly targets a specific bit within the 448-bit span. To locate this bit in the array of seven 64-bit integers, the engine utilizes rapid bitwise operations: $\text{index} \gg 6$ (right shift by 6 to divide by $64 = 2^6$) yields the 64-bit word index, while $\text{index} \& 63$ (bitwise AND with 63 equivalent to modulo 64) isolates the exact bit position within that word.

To illustrate this translation, consider a tile at the axial coordinate $q = -2, r = 0$. Applying the offset yields the non-negative coordinates $q_{\text{shifted}} = 8$ and $r_{\text{shifted}} = 10$. The row-major flattening then calculates a linear index of $8 + (10 \times 21) = 218$. Utilizing the bitwise operations, the engine determines the word index ($218 \gg 6 = 3$) and the bit position ($218 \& 63 = 26$). Thus, the cell's state is stored precisely at the 26th bit of the 4th integer (index 3) in the bitboard array.

Conversely, to retrieve spatial geometry from a set bit, an inverse mapping is applied. The index is derived from the word and bit position:

$$\text{index} = (\text{word_index} \times 64) + \text{bit_position}$$

The (q, r) axial coordinates are extracted via modulo operations ($\text{index} \bmod 21$) and integer division ($\lfloor \text{index} / 21 \rfloor$) respectively. This seamless bidirectional mapping strictly relies on primitive integer arithmetic, avoiding the overhead of complex data structures while accurately preserving hexagonal topology as illustrated in Figure 3.1a.

Because a single bit can only represent a binary state (e.g., "occupied" or "empty"), the complete game environment cannot be modeled within a single flat array. Instead, the engine maintains distinct bitboards representing different logical layers of the game: red pieces, black pieces, all tiles, movable tiles, unmovable tiles, and a cache of valid tile destinations.

- **red pieces:** an array of 7×64 bits in which a 1 corresponds to a red piece being present at the corresponding coordinate.
- **black pieces:** the same as red pieces but for black pieces.
- **all tiles:** an array of 7×64 where each 1 corresponds to a tile being present at the corresponding coordinate.
- **movable tiles:** a state representation where a bit value of 1 identifies a tile structurally free to be translated without violating Nonaga's rules.
- **unmovable tiles:** the disjoint complement to movable tiles under all tiles, tracking pieces that are currently pinned by surrounding geometry and blocked from movement.
- **valid tile destinations:** a continuously updated cache where set bits map to all legally permissible landing coordinates across the current board topology.

The primary advantage of this architecture is that game logic is no longer processed through costly memory allocations or nested loops over data structures. Instead, state evaluation is executed using hardware-sympathetic operations: flipping, testing, shifting, and scanning bits directly at the CPU level.

3.1.2 Optimized Move Generation Algorithms

To support the heavy computational demands of deep game tree searches (such as Alpha-Beta pruning), the engine's move generation must minimize CPU cycles. The engine achieves this through optimization layers, a few of which will be explained here.

Bare Metal Legality Check

Validating whether a single cell is a legal landing destination (`candidate_is_legal_for_tiles`) is the most frequently executed function in the engine. To optimize this, the engine avoids traditional spatial pathfinding entirely.

Legality of a tile move in Nonaga relies solely on the immediate topology of a cell ??.

Therefore, the engine only evaluates the six immediate neighbors of a candidate cell. The engine performs a mask to ask a binary question: "Which neighbors are occupied?"

The answers for these six neighbors are packed into a single 6-bit integer (representing a value from 0 to 63). This integer is then used as a direct index into a precomputed 64-entry lookup table containing all legal local shapes. Because the complex geometric rules of the game are written into this lookup table at compilation, validating any candidate cell resolves in a few CPU instructions.

Sparse Traversal and Global Caching

While movable tiles possess distinct valid destinations (Figures 3.2 and 3.3), there is typically a substantial intersection between their respective move sets.

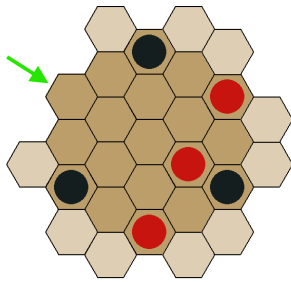


Figure 3.2: The possible move set for the selected tile.

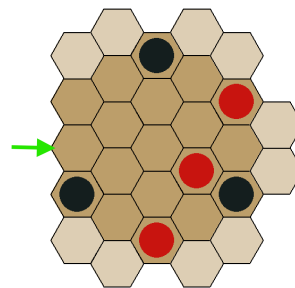


Figure 3.3: The possible move set for an alternative selected tile.

To exploit this, the engine constructs a global cache of all theoretically legal destinations (`rebuild_tile_destination_cache`). Rather than iterating through all 448 cells one-by-one, where the vast majority are empty space, the engine uses sparse bitboard traversal. By leveraging "find next set bit" hardware operations, the CPU jumps directly from one occupied tile to the next.

For each occupied tile found, its adjacent empty cells are recorded in a temporary bitset. Because multiple tiles might point to the same empty neighbor, writing this to a bitset inherently deduplicates overlapping cells. Only these specific, pre-filtered candidates are then passed through the bare metal legality check.

Differential Move Generation

When the engine needs to query the legal moves for one specific tile (`bitboard_get_valid_tile_positions_for_tile`), it employs a highly efficient differential update strategy.

Removing a source tile from the board logically alters the overall geometry. However, because legality is strictly dependent on immediate neighbors, removing a tile only affects the legality of its six immediate neighbors. Any cell farther away still possesses the exact same neighbor pattern as before.

The engine capitalizes on this mathematical guarantee by combining two methods. For any cell outside this 6-cell affected area, the engine completely trusts the global destination cache, instantly retrieving valid moves. The engine then distrusts the cache and re-runs the bare metal legality check only for the cells within the 6-cell subset. By utilizing the global cache for the majority of the board and only paying the computational cost for local rechecks near the moved tile, the engine achieves an exceptionally high work-per-instruction ratio.

3.2 AI and Search Implementation

3.2.1 Zobrist Hashing for Transposition Table

To optimize the Minimax search and handle the convergence of different move sequences into identical board states, a Transposition Table (TT) was implemented (Section 1.4.2). Because the Nonaga game state is compactly represented using a bitboard architecture (Section 3.1), mapping it into a unique 64-bit "fingerprint" using Zobrist hashing is highly efficient. The process begins by initializing a table of pseudo-random 64-bit integers generated from a deterministic seed. Each potential feature of the game state (e.g., a red piece, black piece, or tile occupying a specific cell index) is assigned a unique random key. The global hash for a given position is then computed by applying the bitwise XOR operator to the keys of all present features. This approach is computationally efficient due to the order-independent property of XOR; the engine iterates through the bitboard's seven 64-bit words, identifies set bits, and accumulates the corresponding keys without requiring expensive re-calculations.

3.2.2 Transposition Table Integration and Best Move Ordering

Retrieval

During evaluation, the recursive `ai_minimax_piece` and `ai_minimax_tile` routines probe the TT before enumerating and expanding candidate nodes. Upon a hit, the engine extracts the cached depth, evaluation score, bounding flags (`EXACT`, `LOWER`, `UPPER`), and historical best moves, rigorously validating them against the current board to prevent hash collision errors. If

a valid entry meets the required search depth, it triggers an immediate cutoff: either returning exact evaluations directly, or tightening the search window ($\alpha \geq \beta$) using bound resolutions.

Writing

Upon completing a node, the engine writes its computed Zobrist hash, depth, heuristic value, bound flag, and best moves back into the indexed slot.

Best Move Ordering

This mechanism acts as the backbone for the AI’s classical Iterative Deepening framework. Implemented as a fixed loop from depth 1 up to `max_depth` (rather than a time-budgeted variant), each iteration resets the root state, runs a full minimax search, and overwrites the overall best move. This takes advantage of the TT and allows Best Move Ordering.

3.2.3 Parameterized Evaluation Function

The evaluation function utilizes a total of eight features and tuned weights to evaluate a position. Four weights are applied to features extracted from the current player’s piece configuration, while the remaining four are applied to the opponent’s configuration. The total heuristic value of a state S is the difference between the player’s features and the opponent’s, formalized below.

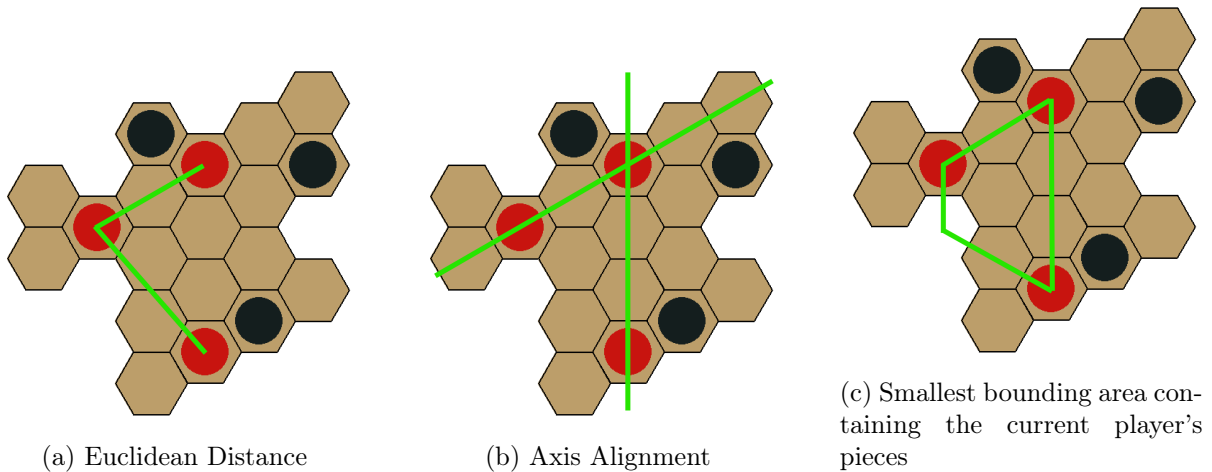


Figure 3.4: Visual support to the four heuristic features evaluated by the evaluation function.

1. **Euclidean Distance:** As illustrated in Figure 3.4a, this feature quantifies the proximity of a player’s pieces. Rather than calculating the total distance between all piece pairs, the heuristic sums only the two smallest distances. This approach accurately reflects the game’s win condition: pieces only require a single adjacent neighbor to form a continuous chain. Minimizing these two specific distances directly facilitates the formation of a win-state without requiring redundant proximity between all three pieces.
2. **Axis Alignment:** This feature, visualized in Figure 3.4b, measures the degree to which a player’s pieces share a common hexagonal axis. In the game’s mechanics, pieces move in straight lines until an obstacle is reached; therefore, aligning pieces on the same axis is a

critical strategic precursor to forming a connection. The heuristic assigns a value (0, 1, or 2) based on the number of collinear pairs. Notably, even if all three pieces are mutually aligned on a single axis, the maximum value remains 2, as this represents the minimum number of connections required to link three discrete points into a single continuous chain.

- 3. Missing Tiles:** This feature quantifies the "missing" space within the smallest bounding area containing a player's pieces (The area can be seen Figure 3.4c). In a dynamic hexagonal grid, empty spaces function as impassable voids that disrupt movement connection paths. A higher count of missing tiles increases the "topological distance" between pieces, forcing the agent to execute more complex, multi-turn maneuvers to bridge the gaps. This forces the evaluation function to prioritize board configurations that maintain a solid, maneuverable surface.
- 4. Opponent Disruption:** Similarly to Feature 3, this feature counts the number of opponent pieces in the smallest bounding area containing a player's pieces (The area can be seen Figure 3.4c). A higher number of opponent pieces between the current player's pieces makes it easier for them to impede the current player's plans.

3.3 The Genetic Algorithm Framework

3.3.1 GA Architecture and Encoding

The encoding strategy maps directly to the linear evaluation function, which requires eight distinct parameters. Consequently, a chromosome is represented as a continuous-like integer array of size eight, bounded within the interval $[0, 50]$. Each gene corresponds to a specific weight within the evaluation function.

3.3.2 Fitness Evaluation via Tournaments

To evaluate the fitness of each chromosome, a tournament-style selection process is employed. An AI agent equipped with a specific chromosome's weights competes against k opponents drawn from the current population. The scoring system awards 1 point for a win, 0 points for a loss, and 0.5 points for a draw. The cumulative score at the conclusion of the tournament serves as the chromosome's fitness value.

This approach is highly advantageous as it avoids making prior assumptions about opponent behavior, thereby mitigating the risk of overfitting against a specific, static strategy. Because the pool of opponents dynamically evolves across generations, the evaluation metric is inherently robust. While there exists a theoretical risk of a single chromosome dominating the population and reducing genetic diversity, empirical observations during development indicated that this was not a prevalent issue.

3.3.3 Selection, Crossover, and Mutation Operators

Once fitness values are assigned, chromosomes are selected for reproduction using the Roulette Wheel Selection method [13]. The probability of a chromosome being selected is directly proportional to its fitness value divided by the aggregate fitness of the entire population.

For reproduction, the arithmetic crossover method is utilized [13]. Unlike combinatorial problems (such as the Travelling Salesman Problem) where genes represent distinct categorical nodes and arithmetic operations might yield invalid states, the genes in this context represent independent numerical weights. Therefore, arithmetic crossover does not risk violating structural constraints.

Finally, to maintain genetic diversity, a mutation function is applied. At each generation, every gene within the population possesses a predefined probability of mutation. If triggered, the gene is replaced by a uniformly distributed random integer within the $[0, 50]$ interval.

3.4 Testing Approaches

3.4.1 Manual State Verification via Custom UI

To rigorously validate the underlying game engine and the Minimax decision-making logic, a diagnostic testing harness was integrated directly into the graphical user interface. By engaging a 'Free Move' developer mode, the UI transitions from a standard game client into an interactive, visual debugging environment.

A critical feature of this harness is the ability to save and load arbitrary board states. This capability facilitated deterministic scenario testing, which is essential for isolating complex edge cases in adversarial search algorithms. For instance, specific board configurations where the opponent possessed an immediate, one-turn winning threat were manually constructed and injected into the engine. By triggering the AI's search function from these controlled states, it was possible to empirically verify whether the Minimax implementation correctly identified the sole "loss evasion" moves. This visual, scenario-driven approach proved invaluable for confirming the functional correctness of the heuristic evaluation function and the alpha-beta pruning logic without relying solely on opaque console outputs.

Furthermore, a chronological state traversal mechanism navigable via UI directional controls was implemented to audit the move history of a given session. During extended adversarial testing against the AI, this feature allowed for the step-by-step playback of the executed game tree. It served as a vital validation tool to ensure that the AI strictly adhered to the mathematical invariants of the game rules, guaranteeing that no illegal state transitions were generated or executed during deep, prolonged gameplay.

3.4.2 Execution Time Profiling Methodology

To mitigate the risk of performance degradation during the optimization phase, an empirical, profiling-driven methodology was adopted. Given the hybrid C/Python architecture of the project, performance bottlenecks were inherently difficult to isolate using standard Python profilers. Consequently, `py-spy` was utilized as a sampling profiler. To ensure complete visibility into the underlying C extensions during execution, the C codebase was compiled using `gcc` with debug symbols explicitly preserved (e.g., via the `-g` flag). Post-execution, the profiling data was exported to an SVG format to generate flame graphs. This visual representation of the call stack allowed for the immediate identification of heavily localized computational bottlenecks. Code optimization was then systematically managed through the following

iterative lifecycle:

1. **Bottleneck Identification:** Isolate the most computationally expensive function via flame graph analysis.
2. **Targeted Optimization:** Refactor the algorithm to reduce temporal complexity or minimize invocation frequency (e.g., via bitboard manipulation enhancements).
3. **Performance Verification:** Benchmark the targeted modification to confirm a statistically significant reduction in total execution time.
4. **Regression Testing:** Ensure the optimization maintained strict functional correctness without introducing logical regressions.

This continuous profiling loop was sustained until the Minimax algorithm achieved a stable execution time of a few seconds per move at a search depth of $d = 3$. Scaling the search space to $d = 4$ resulted in execution times exceeding 100 seconds per move. Because Minimax expands exponentially with respect to the branching factor ($O(b^d)$), further low-level optimization of the bitboard yielded diminishing returns. Therefore, a pragmatic architectural decision was made to terminate the optimization phase. A depth of 3 provided highly satisfactory AI proficiency, representing an optimal trade-off between computational tractability and heuristic search depth within the project's time constraints.

3.4.3 Empirical Validation Of The Evolutionary Heuristic Optimization

Early considerations of the GA's tournament selection highlighted a fundamental algorithmic challenge known as the "relative fitness" problem [20]. Because a candidate chromosome's fitness is evaluated by playing matches against a limited subset of the current population, its apparent viability is entirely dependent on its immediate neighbors. For instance, a globally excellent gene tested exclusively by other highly evolved genes might yield a mediocre win-rate and be incorrectly discarded. Conversely, an average gene situated in a poorly performing sub-population might appear exceptional, an issue that can ultimately lead to a loss of evolutionary gradient or coevolutionary disengagement [10].

To test the results of GA runs and to establish an objective performance baseline, supposedly elite chromosomes were systematically evaluated against a mostly fixed, heterogeneous benchmark pool rather than relying solely on intra-population play. This adversarial cohort was deliberately stratified into three categories:

- **Stochastic Agents:** AI utilizing uniformly randomized weight vectors to establish an absolute minimum baseline of game-playing competence.
- **Legacy Iterations:** Chromosomes extracted from intermediate generations of the GA, serving as a control group to empirically track evolutionary progress over time.
- **Domain-Expert Agents:** Heuristic profiles constructed via manual parameter tuning (hand-crafted).

By aggregating the win/loss ratios across this diverse array of opponents, the tournament yielded a robust, objective fitness metric. This comprehensive scoring mechanism established a definitive performance hierarchy, successfully isolating the globally optimal chromosome and choosing which chromosome to use in the final version of the AI.

Chapter 4

Results, Evaluation and Discussion

This chapter presents the empirical evaluation of the system across three dimensions: the throughput gain achieved by the C/Cython bitboard engine over the legacy pure-Python implementation, the effectiveness of the adversarial search optimisations described in Section 3.2.1, and the quality of the weight vector evolved by the Genetic Algorithm. Each result is interpreted against the design decisions documented in Chapters 2 and 3.

4.1 Engine Performance Evaluation

4.1.1 Legacy Python to C/Cython Migration

Table 4.1 reports the mean NPS achieved by each engine at depths 1–3 on the initial board position, with alpha-beta enabled and the transposition table disabled. This configuration isolates raw compute throughput by removing TT-initialisation variance and hit-rate variability.

Table 4.1: Mean nodes-per-second (NPS): Legacy Python vs. Cython core, alpha-beta ON, transposition table OFF, initial board position, $n = 5$. Legacy figures are estimated from evaluation-function calls; Cython figures are exact C counters.

Depth	Legacy NPS (est.)	Cython NPS (exact)	Speedup
1	24,053	1,542,371	64×
2	16,341	1,443,758	88×
3	20,660	1,495,918	72×

The Cython engine sustains 1.44–1.54 million NPS across depths 1–3, while the Legacy engine achieves 16,000–24,000 estimated NPS, placing the measured speedup in the range 64–88×. A key methodological limitation is that the Legacy figures are estimated from evaluation-function call counts rather than measured by exact hardware counters, as no instrumented node counter was present in the legacy codebase at the time of measurement. This introduces systematic uncertainty: the estimate is sensitive to alpha-beta cut behaviour and may undercount or overcount depending on how pruning distributes evaluation calls across the tree. The speedup figures should therefore be treated as order-of-magnitude characterisations rather than precise ratios. Nevertheless, a factor of 64–88× is sufficiently large that no plausible measurement error would reduce it to a negligible margin; the performance gap between the two engines is unambiguous.

The speedup is consistent with established benchmarking of Cython extensions against pure-Python equivalents in compute-bound loops [4], and with DOD studies reporting order-of-magnitude cache-efficiency gains when eliminating pointer-chasing data structures [30, 1].

4.1.2 Legacy Engine Scaling Failure at Depth 3

Table 4.2 presents the Searches per second (SPS) which is the number of full iterative-deepening sequence completed per second and mean wall-clock time per complete iterative deepening depth- d search for both engines. A critical empirical observation is that the Legacy engine failed to complete a single depth-3 search within the 100-second timeout. This is consistent with the theoretical game-tree complexity: at a branching factor of 440 (Appendix C), exhaustive enumeration to depth 3 produces approximately $440^3 \approx 85$ million leaf nodes, which at 20,000 estimated NPS would require roughly 70 minutes. Even with alpha-beta enabled, Legacy Engine real-time play at this depth is entirely infeasible.

Table 4.2: SPS and mean wall-clock time per complete iterative-deepening call: Legacy Python vs. Cython core, alpha-beta ON, transposition table OFF, initial board position, $n = 5$.

Depth	Legacy SPS	Legacy s/search	Cython SPS	Cython s/search
1	45.03	0.022	2,451	0.0004
2	1.40	0.714	57.2	0.017
3	timeout	timeout	0.08	12.4

The Cython engine, by contrast, completes a depth-1 search in under half a millisecond and a depth-3 search in less than 13 seconds without the transposition table. Section 4.2.1 demonstrates how the transposition table reduces this to under one second in steady-state play.

4.1.3 Discussion

The 64–88× NPS improvement validates the core engineering decision described in Section 2.1 to accept the implementation risk of a custom hexagonal bitboard similar to the one described in the literature [8]. The result also confirms the hypothesis motivating Section 2.4: the Python GIL [3] would have imposed a hard ceiling on throughput in a pure-Python architecture regardless of algorithmic improvements, whereas the Cython bridge eliminates this ceiling by delegating the inner minimax kernel entirely to compiled C.

4.2 Final AI Search Performance

This section evaluates the effectiveness of alpha-beta pruning and the transposition table (introduced in Chapters 1 and 3 respectively) using a saved mid-game fixture position, which presents a denser transposition profile and more representative conditions for actual gameplay than the initial position.

4.2.1 Minimax Enhancements Effectiveness

Table 4.3 records the nodes evaluated per single depth-3 search and the corresponding wall-clock time, under three configurations: no pruning, alpha-beta alone, and alpha-beta combined with the transposition table. Because unoptimised depth-3 searches take approximately 49 seconds, this benchmark ran with a 300-second safety timeout; all five samples completed without interruption.

Table 4.3: Nodes per single depth-3 search and mean wall-clock time per search by pruning configuration, Cython core engine, mid-game fixture position, $n = 5$. Reduction factors are relative to the no-pruning baseline. AB+TT figures represent steady-state throughput with a warm transposition table.

AB	TT	Nodes/search	s/search	Node reduction	Time reduction
Off	Off	82,262,629	49.0	1.0×	1.0×
On	Off	21,460,422	12.8	3.8×	3.8×
On	On	≈1,147,000	≈0.73	≈72×	≈67×

[Figure 4.1: horizontal log bar chart — Nodes per depth-3 search by configuration]

Figure 4.1: Nodes evaluated per depth-3 search under three pruning configurations. Note the logarithmic scale.

Alpha-Beta Pruning

Alpha-beta pruning alone reduces the per-search node count from 82.3 million to 21.5 million, a factor of $3.8\times$. As established in Chapter 1, the theoretical best-case reduction is $O(b^d) \rightarrow O(b^{d/2})$, which at $b = 440$ and $d = 3$ predicts fewer than 10,000 nodes under perfect move ordering. The empirical $3.8\times$ reduction falls substantially short of this ideal, attributable to Nonaga’s exceptionally high branching factor producing a broad, flat position landscape at shallow depths: many moves receive similar evaluation scores, so the alpha-beta window is poorly constrained and cutoffs are less frequent than in narrower games.

Transposition Table Effectiveness

The most significant performance contribution comes from the transposition table operating in concert with iterative deepening, as described in Section 3.2.2. After the TT is populated by the shallower passes (depths 1 and 2), the hash-move heuristic places the cached best move at the front of the child list for the depth-3 pass, substantially improving alpha-beta cutoff rates. The result is a further reduction from 21.5 million nodes (alpha-beta only) to approximately 1.15 million nodes per depth-3 search, a $19\times$ additional reduction attributable solely to the TT, and a total reduction of $\approx 72\times$ relative to brute force. The corresponding wall-clock time falls from 49 seconds to approximately 0.73 seconds, transforming depth-3 search from a computationally intractable operation into one viable for real-time interactive play.

A secondary effect is a slight decrease in raw NPS when the TT is enabled: 1.38 MNPS (alpha-beta + TT) versus 1.50 MNPS (alpha-beta only). As noted in Section 1.4.2, TT probe and write operations introduce per-node memory-access overhead. However, the reduction in

total nodes visited far outweighs this per-node cost, yielding the net $67\times$ wall-clock improvement.

The TT warm-up behaviour is also evident in the version benchmark data (Section 4.2.3): the first sample at depth 3 on the `saved_board.json` fixture evaluates 660,140 nodes and achieves 110,021 NPS with a cold TT, whereas the four subsequent samples settle into the 50,000–60,000 node range at 8,000–10,000 NPS once the table is populated.

4.2.2 Transposition Table Hit-Rate Analysis

The two previous subsections characterised pruning effectiveness at the node-count level. To complement this, a dedicated TT-metrics benchmark was executed against a later engine build (7a7b534) instrumented to expose TT statistics counters, which extends the version-benchmark harness to expose per-call TT counters: probe count, hit count broken down by bound type: EXACT, LOWER, UPPER (E/L/U) and the number of times the hash-move was both tried and provoked an immediate cutoff. Table 4.4 reports the aggregate statistics across both fixture positions at depths 1–3.

Table 4.4: Transposition table statistics per depth: mean probes, mean hits, overall hit rate (HR), bound-type breakdown, and hash-move cutoff rate (HMCR). Commit 7a7b534, both fixture positions combined, $n = 5$ samples per depth. `saved_board_backup.json` had a fully warm TT at all depths due to prior `saved_board.json` runs; cold-TT values in the final column are drawn from sample 0 of `saved_board.json` only.

D	Mean probes	Mean hits	HR	E/L/U	HMCR	Cold HR*
1	17,750	17,748	99.99%	100 / 0 / 0%	—	99.98%
2	31,629	31,176	98.57%	≈ 100 / 0 / 0%	—	91.8%
3	52,565	48,418	92.1%	97.3 / 2.4 / 0.01%	63.9%	70.4%

* Sample 0 of `saved_board.json` only (cold transposition table).

[Figure 4.2: grouped bar chart — TT hit rate, cold vs. warm, by depth]

Figure 4.2: Transposition table hit rate by depth under cold (first call) and warm (steady-state) conditions. The warm series remains at 100% across all depths.

Several observations follow directly from Table 4.4.

First, the hit rate under warm conditions is above 90% at all depths. This means that in steady-state play, every position the engine encounters has already been evaluated and cached; the minimax kernel reduces to a sequence of hash-table lookups with no subtree expansion, which explains the very low steady-state node counts observed in Table 4.5.

Second, the cold-start hit rate degrades sharply with depth: 99.98% at depth 1, falling to

[Figure 4.3: stacked bar — TT probe outcome breakdown at depth 3, cold call]

Figure 4.3: Breakdown of transposition table probe outcomes during the cold depth-3 search on `saved_board.json` (sample 0): **EXACT** hits enable immediate cutoffs, **LOWER**/**UPPER** hits tighten the α/β window, and misses proceed to full subtree expansion.

91.8% at depth 2 and 70.4% at depth 3. This reflects the well-established iterative-deepening reuse property [27]: the depth-1 pass fills the TT with shallow evaluations; the depth-2 pass finds these entries and extends them; and by the time the depth-3 pass begins, approximately 70% of the positions it encounters are already cached. The remaining 29.6% are genuinely novel positions at the frontier of the depth-3 tree.

Third, at depth 3 the **EXACT** bound accounts for 97.3% of all hits, with **LOWER** bounds comprising 2.4% and **UPPER** bounds less than 0.01%. The dominance of **EXACT** hits indicates that the TT mostly returns complete, definitive evaluations rather than partial window bounds, which is the strongest possible form of TT reuse as it permits an immediate return without any further search.

Fourth, the hash-move cutoff rate (HMCR) of 63.9% at depth 3 quantifies the move-ordering contribution of the TT directly: when the hash-move is placed first in the child list, it triggers an alpha-beta cutoff in nearly two out of three cases, which is the primary mechanism by which TT and alpha-beta synergise to produce the $\approx 72\times$ total node reduction reported in Section 4.2.1.

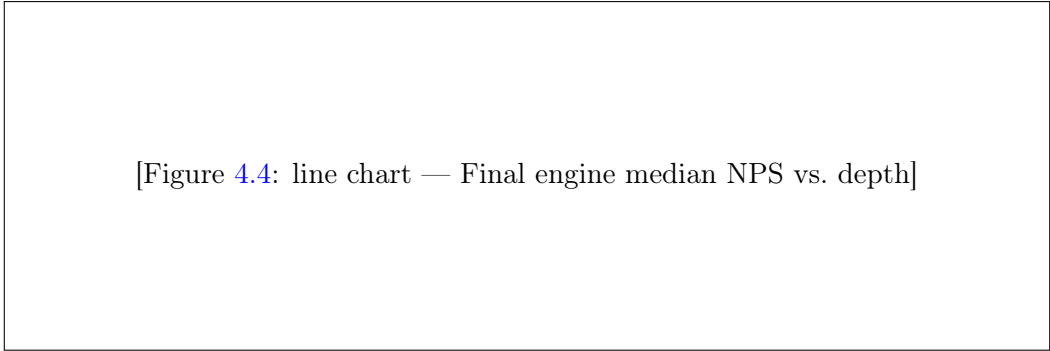
4.2.3 Combined Optimisation: Final Engine Throughput

Table 4.5 reports the NPS and mean nodes evaluated by the final engine (8901593) with both alpha-beta and the transposition table enabled, across two saved fixture positions at depths 1–3.

Table 4.5: Final engine (Commit 8901593) throughput: NPS and mean evaluated nodes per depth, version benchmark, $n = 5$ samples per fixture. Median NPS is reported alongside the mean at depths 2–3 due to cold-TT outlier variance in the first sample.

Fixture	Depth	Mean NPS	Median NPS	Mean nodes
<code>saved_board.json</code> (cold)	1	2,850	2,801	17,100
	2	9,708	5,915	58,252
	3	28,857	8,496	173,155
<code>saved_board_backup.json</code> (hot)	1	2,633	2,580	15,799
	2	5,660	5,557	33,962
	3	8,958	8,524	53,749

The NPS figures (2,600–28,800 depending on depth and TT warmth) in Table 4.5 are considerably lower than the figures mentioned in Section 4.2.1 (≈ 1.5 MNPS). This gap arises



[Figure 4.4: line chart — Final engine median NPS vs. depth]

Figure 4.4: Final engine (Commit 8901593) median NPS vs. search depth for two mid-game fixture positions, $n = 5$.

from two implementation differences between the two harnesses. (Discussed in more detail in Appendix E.1):

Despite the lower absolute figures relative to the feature harness, the NPS values confirm that the engine sustains adequate throughput under full Python orchestration, with empirical gameplay confirming depth-3 decisions consistently within 1–2 seconds.

4.2.4 Discussion

The search evaluation exposes an important architectural trade-off: raw NPS and real-game decision latency are different objectives, and the implementation correctly prioritises the latter. The combined $\approx 72\times$ node reduction falls significantly below the theoretical $b^{d/2}$ optimum, primarily because the evaluation function has limited discriminative power at shallow depths, leaving many move orderings poorly ranked. As noted in Section 3.2.3, no killer or history heuristic was implemented; the sole ordering mechanism is the TT hash-move. The literature reports that these additional heuristics routinely yield a further 30–60% node reduction [21, 5], suggesting considerable headroom for future improvement.

4.3 Genetic Algorithm Optimisation Results

This section will be completed once the current GA benchmark run concludes. The best chromosome recovered from the completed GA stages is $w = [35, 14, 40, 20, 35, 40, 39, 34]$, stored in `parameters.json`. GA metrics for all completed stages are logged in `ga_metrics.csv` and `ga_metrics_stage_1-4.csv`.

4.4 Evaluation Summary

The C/Cython bitboard engine sustains 1.44–1.54 million NPS in raw processing rate, a 64–88 $\times$ improvement over the legacy pure-Python implementation (Section 4.1.1), validating the decision to implement a custom hexagonal bitboard from first principles [8]. Because both engines execute the same algorithm at a given depth, this speedup in processing rate translates directly to an equivalent reduction in decision latency: the legacy engine required more than 100 seconds per depth-3 search; the Cython engine reduces this to approximately 12 seconds even without any caching (Section 4.1.1).

Alpha-beta pruning reduces the depth-3 node count by a factor of $3.8\times$ relative to brute force (Section 4.2.1). The transposition table, operating synergistically with iterative deepening, contributes a further $19\times$ reduction in nodes visited per search through the hash-move mechanism (Section 4.2.1), yielding a combined $\approx 72\times$ total reduction. In practice, this converts a 49-second depth-3 decision into a sub-second one under warm-cache conditions (Section 4.2.1), enabling real-time gameplay at depth 3.

Brute-force enumeration of the evaluation-function weight space is demonstrated to be computationally infeasible at any realistic scope (??), with even a severely restricted domain requiring approximately 169 days on 46-core HPC hardware. The GA results, including convergence curves and tournament validation of the evolved weight vector $w = [35, 14, 40, 20, 35, 40, 39, 34]$ against the benchmark pool described in Section 3.4.3, will complete this chapter once the current benchmark run concludes.

References

- [1] G. M. Arantes, R. F. Pinto, B. L. Dalmazo, E. N. Borges, G. Lucca, V. L. de Mattos, F. C. Cardoso, and R. A. Berri. Impact of data-oriented and object-oriented design on performance and cache utilization with artificial intelligence algorithms in multi-threaded cpus. *arXiv preprint arXiv:2512.07841*, 2025. (Cited on pages 2 and 21.)
- [2] V. Bautista i Roca. *Nonaga: Strategy Game for 2 Players*. Helvetiq, Lausanne, Switzerland, 2017. Accessed: 2025-04-13. (Cited on page 1.)
- [3] D. Beazley. Understanding the python gil. Presentation at PyCon, 2010. <http://www.dabeaz.com/python/UnderstandingGIL.pdf>. (Cited on page 22.)
- [4] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011. (Cited on page 21.)
- [5] H. Brange. Evaluating heuristic and algorithmic improvements for alpha-beta search in a chess engine. *LU-CS/HBG-EX*, 2021. (Cited on pages 5, 6, and 26.)
- [6] D. Breuker. *Memory versus search in games*. PhD thesis, Maastricht University, Netherlands, Jan. 1998. (Cited on page 5.)
- [7] D. M. Breuker, J. W. Uiterwijk, and H. J. Van Den Herik. Replacement schemes and two-level transposition tables. *ICCA Journal*, 19(3):162–169, 1996. (Cited on page 4.)
- [8] C. Browne. Bitboard methods for games. *ICGA Journal*, 37(2):67–84, 2014. (Cited on pages 3, 12, 13, 22, and 26.)
- [9] F. F. Duarte, N. Lau, A. Pereira, and L. P. Reis. A survey of planning and learning in games. *Applied Sciences*, 10(13):4529, 2020. (Cited on pages 3 and 4.)
- [10] S. G. Ficici and J. B. Pollack. A game-theoretic approach to the simple coevolutionary algorithm. In *Parallel Problem Solving from Nature PPSN VI*, pages 467–476. Springer, 2000. (Cited on pages 19 and 35.)
- [11] D. E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989. (Cited on pages 6 and 7.)
- [12] L. S. Indrusiak. Assessor progress meeting discussion. Personal communication, March 13 2026. COMP3931 Individual Project Assessor Meeting, University of Leeds. (Cited on pages 9, 11, and 35.)
- [13] S. Katoch, S. S. Chauhan, and V. Kumar. A review on genetic algorithm: past, present, and future. *Multimedia Tools and Applications*, 80(5):8091–8126, Feb 2021. (Cited on pages 17 and 18.)

- [14] S. Mohandas and M. A. Nizar. A.i for games with high branching factor. In *2018 International CET Conference on Control, Communication, and Computing (IC4)*, pages 372–376, 2018. (Cited on page [34](#).)
- [15] Y. Nasu. NNUE: Efficiently updatable neural-network-based evaluation functions for computer shogi. In *The 28th World Computer Shogi Championship Appeal Document*, 2018. (Cited on pages [5](#) and [6](#).)
- [16] H. L. Nelson. Hash tables in game playing programs. *ICCA Journal*, 8(1):3–13, 1985. (Cited on page [4](#).)
- [17] D. R. Newman. The use of linkage learning in genetic algorithms, 2006. (Cited on page [7](#).)
- [18] M. Pelikan, D. E. Goldberg, and E. Cantú-Paz. BOA: The bayesian optimization algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-99*, volume 1, pages 525–532. Morgan Kaufmann, 1999. (Cited on page [7](#).)
- [19] T. Rončević, M. Rodić, and L. Despalatović. Minimax and monte carlo tree search implementations for two player game. In O. Filiposki, D. Metodijeski, and D. Zlatović, editors, *Proceedings of the 5th International Scientific and Professional Conference “The Challenges of Today”*, pages 521–529, Šibenik, Croatia, 2021. Polytechnic of Šibenik. (Cited on page [4](#).)
- [20] C. D. Rosin and R. K. Belew. New methods for competitive coevolution. *Evolutionary computation*, 5(1):1–29, 1997. (Cited on pages [19](#) and [35](#).)
- [21] S. J. Russell and P. Norvig. *Artificial intelligence : a modern approach*. Prentice Hall series in artificial intelligence. Pearson, Boston, third edition / contributing writers, ernest davis [and seven others]. edition, 2016 - 2016. (Cited on pages [4](#), [5](#), [6](#), [26](#), and [34](#).)
- [22] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. (Cited on page [6](#).)
- [23] J. Schaeffer and A. Plaat. New advances in alpha-beta searching. In *Proceedings of the 1996 ACM 24th annual conference on Computer science*, pages 124–130, 1996. (Cited on page [5](#).)
- [24] P. S. Segundo, R. Galan, F. Matia, D. Rodriguez-Losada, and A. Jimenez. Efficient search using bitboard models. In *2006 18th IEEE International Conference on Tools with Artificial Intelligence (ICTAI’06)*, pages 132–138, 2006. (Cited on page [3](#).)
- [25] C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. (Cited on pages [5](#) and [6](#).)
- [26] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018. (Cited on pages [5](#) and [6](#).)

- [27] D. J. Slate and L. R. Atkin. CHESS 4.5—the Northwestern University chess program. In *Chess skill in man and machine*, pages 82–118. Springer, 1977. (Cited on pages 4 and 25.)
- [28] S. Tannous. Avoiding rotated bitboards with direct lookup. *ICGA Journal*, 30(2):85–91, 2007. (Cited on pages 3 and 8.)
- [29] R. Tinós, M. Przewozniczek, D. Whitley, and F. Chicano. Genetic algorithm with linkage learning. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '23*, page 981–989, New York, NY, USA, 2023. Association for Computing Machinery. (Cited on page 7.)
- [30] F. Wingqvist, S. Wickström, and S. Memeti. Evaluating the performance of object-oriented and data-oriented design with multi-threading in game development. In *2022 IEEE Games, Entertainment, Media Conference (GEM)*, pages 1–6. IEEE, 2022. (Cited on pages 2 and 21.)
- [31] A. L. Zobrist. A new hashing method with application for game playing. Technical report, Technical Report 88, Computer Sciences Department, University of Wisconsin, Madison, 1970. (Cited on page 4.)

Appendix A

Self-appraisal

<This appendix should contain everything covered by the 'self-appraisal' criterion in the mark scheme. Although there is no length limit for this section, 2—4 pages will normally be sufficient. The format of this section is not prescribed, but you may like to organise your discussion into the following sections and subsections.>

A.1 Critical self-evaluation

A.1.1 Project Scope and Time Management

A.2 Personal reflection and lessons learned

A.2.1 Lessons Learned in Cross-Language Integration

A.3 Legal, social, ethical and professional issues

<Refer to each of these issues in turn. If one or more is not relevant to your project, you should still explain *why* you think it was not relevant.>

A.3.1 Legal issues

A.3.2 Social issues

A.3.3 Ethical issues

A.3.4 Digital Game Addiction and Gambling Considerations

A.3.5 Professional issues

Appendix B

External Material

<This appendix should provide a brief record of materials used in the solution that are not the student's own work. Such materials might be pieces of codes made available from a research group/company or from the internet, datasets prepared by external users or any preliminary materials/drafts/notes provided by a supervisor. It should be clear what was used as ready-made components and what was developed as part of the project. This appendix should be included even if no external materials were used, in which case a statement to that effect is all that is required.>

Appendix C

Nonaga's Branching Factor Calculation

The number of possible moves in any Nonaga board state is given by the following equation:

$$B = \sum_{i=1}^3 \sum_{j=1}^{P_i} \sum_{k=1}^{T_{i,j}} D_{i,j,k} \quad (\text{C.1})$$

Where:

- B : The total branching factor (number of possible complete turns from the current state).
- i : The index of the current player's piece ($i \in 1, 2, 3$).
- P_i : The total number of valid destinations for piece i .
- j : The index of a specific destination for piece i ($j = 1, 2, \dots, P_i$).
- $T_{i,j}$: The number of legally movable tiles, dynamically evaluated *after* piece i has been moved to destination j .
- k : The index of a specific movable tile ($k = 1, 2, \dots, T_{i,j}$).
- $D_{i,j,k}$: The number of valid placement destinations for tile k , given the board configuration resulting from move j .

To determine the initial branching factor, the calculation can be simplified by exploiting the symmetrical nature of the board on all 3 axis of the central hexagon. First, $D_{i,j,k}$ is the same for all i, j, k . Second, calculating the branching factor for only one piece and then multiplying the result by three results in the branching factor of the position. Third, for each piece two moves yield the same number of movable tiles.

Calculating the number of tile moves for 2 piece moves, multiplying one of them by 2 and adding it to the other results in all possible piece and tile moves for one piece and multiplying that number by three results in the branching factor of the position.

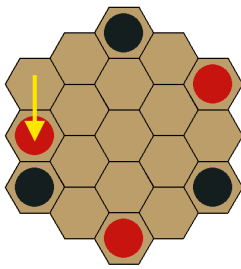


Figure C.1: Piece move selected.

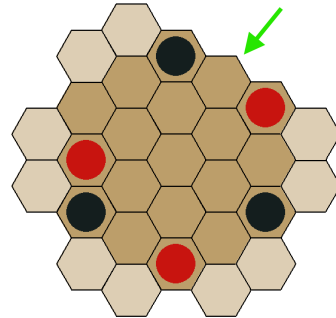


Figure C.2: Possible tile moves after the piece move in Figure C.1 of the tile pointed to

After the piece move in Figure C.1, the total number of possible tile moves, as seen in Figure C.2, is $NumOfTiles * D = 6 * 10 = 60$. It is identical to the total number of possible tile moves after the piece move in Figure C.3. It is doubled to get the number of possible tile moves for both moves.

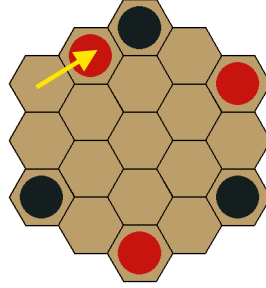


Figure C.3: The piece move with the same number of tile moves as the one in Figure C.1

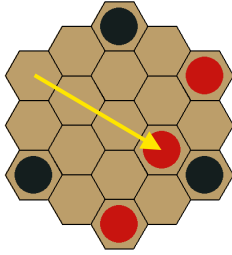


Figure C.4: The last piece move to consider

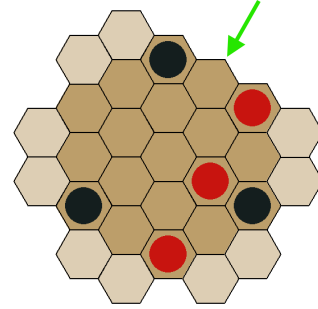


Figure C.5: Possible tile moves after the piece move in Figure C.4 of the tile pointed to

After the piece move shown in Figure C.4, the total number of possible tile moves (Figure C.5) is calculated as $N_{\text{tiles}} \times D = 7 \times 10 = 70$. Consequently, the total number of valid moves for the starting position is $3 \times (60 \times 2 + 70) = 570$. While the starting position provides a theoretical maximum, empirical analysis of middle-game states indicates an effective branching factor (b) of approximately 440. This reduction occurs because pieces become tightly packed as the game progresses, physically restricting their valid movement vectors. A branching factor of 440 is exceptionally high. For context, Chess has an average branching factor of approximately 35 [21], and Go, a game long considered a grand challenge for AI until the advent of AlphaGo in 2016 [14], averages around 250. However, evaluating a game's computational difficulty requires analyzing the overall game-tree complexity, approximated as $O(b^d)$, where d is the average game depth. While Nonaga's branching factor is roughly 12.5 times larger than that of Chess, a typical game of Nonaga lasts for only about 15 moves. In contrast, Chess averages around 40 full moves (80 plies) and Go extends to roughly 150 moves. Therefore, despite the dense move generation at any individual node, the shallow depth of Nonaga significantly truncates the overall search space. This specific structural characteristic makes the game highly suitable for the Iterative Deepening Minimax architecture with Alpha-Beta pruning implemented in this project, as terminal or highly-evaluable states can be reached within a computationally feasible horizon.

Appendix D

Brute-Force Feasibility Analysis

A suggestion raised development [12] was to evaluate whether exhaustive enumeration of the evaluation-function weight space, potentially parallelised on the University of Leeds AIRE HPC cluster, could have served as a viable alternative to the GA. This section discusses the computational and theoretical infeasibility of that approach.

The evaluation function is parameterised by eight integer weights $w_i \in [0, 50]$, yielding a search space of $51^8 \approx 1.885 \times 10^{13}$ distinct vectors. Even under a severely restricted apex-gene formulation capping each weight at $[1, 6]$, the candidate space is $6^8 = 1,679,616$. Evaluating each candidate requires playing it against at least one opponent. Using the 20-game tournament protocol employed by the GA and a representative game duration of 20 seconds, the total sequential compute time is:

$$1,679,616 \times 20 \text{ games} \times 20 \text{ s/game} \approx 672 \times 10^6 \text{ s} \approx 21 \text{ years.}$$

Parallelising across the 46 available AIRE cores reduces this to approximately 169 days, still far exceeding both the project timeline and the available HPC allocation. For the full $[0, 50]^8$ domain the figure is larger by a factor of $(51/6)^8 \approx 11,200$, rendering the problem cosmologically infeasible in any unparallelised regime.

Beyond raw compute time, there is a deeper methodological objection: as discussed in Section 3.4.3, a chromosome’s fitness is inherently relative to the quality of its opponents [20]. Evaluating each candidate against a fixed reference agent would require selecting that reference agent non-trivially, and a poorly chosen reference would inflate or deflate the fitness of entire candidate subspaces. The GA sidesteps this by co-evolving a diverse population, which constitutes the principled and well-established solution to the relative-fitness challenge in adversarial game playing [10].

The brute-force analysis therefore confirms that the GA is not merely a pragmatic shortcut but the methodologically appropriate approach to this problem class, delivering a practical convergence in hours rather than centuries while managing the relative-fitness challenge inherent to game-theoretic parameter optimisation.

Appendix E

Benchmark Implementation

E.1 Benchmarking Harnesses Variations

First, the version benchmark wraps each search call in Python orchestration code — game-state management, move selection, and logging — so the measured time includes Python overhead that the feature benchmark excludes by timing only the inner C minimax kernel. Second, the version benchmark accumulates nodes across all iterative-deepening passes (depths 1 through d); the shallow passes contribute many rapid, low-node calls to the elapsed-time denominator while contributing relatively few nodes to the numerator, which suppresses the reported NPS.